

Practical Data Science (with Python) Assignment 1

Student Name: Aiden Tsui

Student ID: s4005768

Data Preparation

Within the initial dataset “School-Age-Digital-Connectivity-A1data.csvz” we found and fixed a series of errors, this included impossible values, duplicates, input mistakes, missing values, spelling mistakes as well as some others. In order to be able to perform analysis and calculations we had to clean it. By the end of the data cleaning process involving data retrieving and data preparation we have now a cleaned file “cleaned_A1data.csv” which we then used to work through the next stages of the data science process. The data preparation process involved an overlook of the data, finding possible errors that stood out, then using some code to identify some errors and fix them. Below are the errors that were uncovered and the solutions.

Error 1

- Impossible values

Within the data there were a couple years that stood out from the rest, these were impossible years where they were higher than 2023, this meant that the year was impossible. Due to this we could not trust the remaining data for that row and as such dropped the row. This was not a big deal as only two years were impossible and we did not lose too much data. Below we used the drop method, targeting the specific rows with the impossible values.

```
#dropping row with impossible values
data = data.drop(index=65)
data = data.drop(index=51)
```

Error 2

- Duplicates

Using the python code to check for duplicate values it was uncovered that Turkey was displayed 3 times with the exact same values repeated on rows 71 to 73. As such the two extra rows were unnecessary and could cause bias and errors in our data set. Using the same method as with the impossible values we targeted the two specific rows and dropped them.

```
#checking for duplicate rows
duplicates = data[data.duplicated()]
duplicates
```

	ISO3	Countries and areas	Region	Sub-region	Income Group	Total	Rural (Residence)	Urban (Residence)	Poorest (Wealth quintile)	Richest (Wealth quintile)	Data source	Time period
72	TUR	Turkey	ECA	EECA	Upper middle income (UMI)	36%	14%	43%	2%	90%	Demographic and Health Survey	2013
73	TUR	Turkey	ECA	EECA	Upper middle income (UMI)	36%	14%	43%	2%	90%	Demographic and Health Survey	2013

```
#dropping the duplicate rows
data = data.drop(index=72)
data = data.drop(index=73)
```

Error 3

- Human mistakes

Another error was that in the columns “Total” and “Urban (Residence)” one of the values had an extra “%” sign. The extra percentage sign could affect the data when we later turned them into integers to perform calculations. The way we fixed them was by targeting the column which had the error and swapping the values that had “%%” for “%” . We then repeated this process for the other column.

```
#input mistakes(%%)
data.loc[:, 'Total'] = data.loc[:, 'Total'].str.replace('%%', '%')
data.loc[:, 'Urban (Residence)'] = data.loc[:, 'Urban (Residence)'].str.replace('%%', '%')
```

Error 4

- Missing values

Throughout multiple columns and rows we had missing values which appeared as NaN on jupyter notebook. I decided to replace the missing values with the mean, this is so that we don't lose valuable data.. We checked for missing values using the .isna function which helped us count and see where and how many missing values there are. The process of replacing the missing values with the means took a couple steps, first we grouped the columns which had missing values together. Then converted them from objects, stripping the % sign for all data within the respective columns. We then calculated the means and replaced the missing values with the mean. Then we converted them back adding the %”sign. Below you can also see that we rechecked the missing values to make sure they are all filled.

```
#finding missing values
print(data.isna().sum())
```

```
IS03                0
Countries and areas 0
Region              0
Sub-region          0
Income Group        0
Total               0
Rural (Residence)   2
Urban (Residence)   0
Poorest (Wealth quintile) 6
Richest (Wealth quintile) 7
Data source         0
Time period         0
dtype: int64
```

```
#replace missing with mean
columns_missing = ['Rural (Residence)', 'Poorest (Wealth quintile)', 'Richest (Wealth quintile)']

for columns in columns_missing:
    data[columns] = data[columns].str.rstrip('%').astype(float)

column_means = data[columns_missing].mean()
data_cleaned = data.copy()
data_cleaned[columns_missing] = data_cleaned[columns_missing].fillna(column_means)
data_cleaned[columns_missing] = data_cleaned[columns_missing].applymap(lambda x: f"{x:.1f}%")
```

```
#checking missing values
print(data_cleaned.isna().sum())
```

```
IS03                0
Countries and areas 0
Region              0
Sub-region          0
Income Group        0
Total               0
Rural (Residence)   0
Urban (Residence)   0
Poorest (Wealth quintile) 0
Richest (Wealth quintile) 0
Data source         0
Time period         0
dtype: int64
```

Error 5

- Spelling mistakes

Using the “.value_counts()” on the income group we identified that there was an extra variable which occurred due to a spelling mistake. This was with “lowerr” this would have made an extra variable which when graphed or calculated upon would disturb the data. We cleaned this by replacing the spelling error with the correct value.

```
#checking spelling mistakes
data_cleaned['Income Group'].value_counts()
```

```
Upper middle income (UM)    27
Lower middle income (LM)    26
Low income (L)              15
High income (H)             5
Lowerr middle income (LM)    1
Name: Income Group, dtype: int64
```

```
#fixing spelling mistakes
corrections = {'Lowerr middle income (LM)': 'Lower middle income (LM)'}
data_cleaned['Income Group'] = data_cleaned['Income Group'].replace(corrections)
```

Irrelevant observations

Redundant whitespace, there was a case of a redundant whitespace within the data set, this was in row 25 where the input variable ‘Georgia’ had some redundant whitespace, we ended up fixing this by stripping away the white space however since the value was an independent and unique variable the whitespace did not affect the data in anyway and the fix was more for aesthetics. The below is the code we used to remove the whitespace where we removed any whitespace that may be in the dataset.

```
# whitespace
for variables in data.columns:
    data[variables] = data[variables].str.strip() if data[variables].dtype == 'object' else data[variables]
```

Headers, when we first imported the data the headers for the columns were written as column A, column b, column c and so on and the proper headers were underneath. For this we dropped the first two rows and manually imputed the headers. In this way it is easier to read and understand the data where the header matches the corresponding data rather than column a representing ISO3. The below code is importing the data and manually inputting the headers while dropping the two no longer necessary rows.

```
import pandas as pd
#importing data and making the headers
data = pd.read_csv('School-Age-Digital-Connectivity-A1data.csv',header=None, names = ('ISO3',
    'Countries and areas', 'Region', 'Sub-region','Income Group', 'Total', 'Rural (Residence)',
    'Urban (Residence)', 'Poorest (Wealth quintile)', 'Richest (Wealth quintile)',
    'Data source', 'Time period'))
data = data.drop(index=0)
data = data.drop(index=1)
data
```

Some of the extra columns were also dropped in the data cleaning process, this is where the columns provided no insights or had no effect on our analysis.

```
#dropping columns
columns_to_drop = ['Data source', 'Region', 'Sub-region']
data = data.drop(columns=columns_to_drop)
```

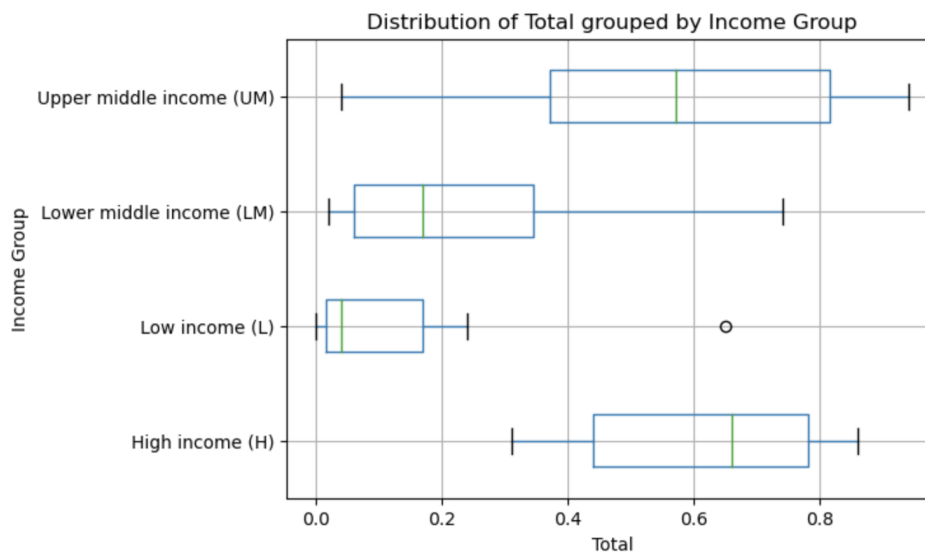
Data Exploration

Task 2.1

Below is a side by side boxplot where the data column “total” has been split into the four subgroups Upper middle income (UM), Lower middle income (LM), Low income (L), High income (H). The data shows a clear separation between the low and lower income groups compared to the upper and high income groups. The median values shown correlates to the income groups where we expect lower income groups to have a lower total however the data shows the upper middle income group has a higher max and upper quartile than the high income group. Another thing to point out is the outlier on the low income group. Below the box plots we also found the means for total by income group, in which they are also ordered as expected, also matching what was observed by the box plots.

```
#side by side boxplot for Total grouped by Income Group
import matplotlib.pyplot as plt
data.boxplot(column='Total', by='Income Group', vert=False)
plt.title('Distribution of Total grouped by Income Group')
plt.xlabel('Total')
plt.ylabel('Income Group')
plt.suptitle('')
```

```
Text(0.5, 0.98, '')
```



```
#mean for each Income Group in total
group = ['Income Group', 'Total']
collective = data[group]
mean_income = collective.groupby('Income Group')['Total'].mean()
mean_income
```

```
Income Group
High income (H)      0.610000
Low income (L)       0.117333
Lower middle income (LM)  0.227037
Upper middle income (UM)  0.560370
Name: Total, dtype: float64
```

Task 2.2

Below we compared rural and urban residence. The median has been calculated where for rural it sits at 13.5% and 40.5%, this value is quite low where even for urban the percentage of school-age children who have internet connection at home is less than half. The top ten countries or areas for urban and rural areas have also been printed out. The highest for rural is The Dominican Republic at 100% and for urban Serbia is at the top at 97%. This comes as a surprise as the general assumption is that urban regions are better off in terms of internet. As shown by the following countries, where rural can be seen as lower when compared to urban.

This could show that just the republic of congo could be an outlier or a special case. Another trend that can be observed is that many of the countries in the top ten appear in both urban and rurals top ten, this could show the wealth of the country.

```
#median for rural
median_rural = data['Rural (Residence)'].median()
median_rural = median_rural * 100
median_rural
```

13.5

```
#median for urban
median_urban = data['Urban (Residence)'].median()
median_urban = median_urban * 100
median_urban
```

40.5

```
#top ten for rural
top_ruralcountries = data.nlargest(10, 'Rural (Residence)')
columns_to_select = ['ISO3', 'Countries and areas', 'Rural (Residence)']
top_ruralcountries = top_ruralcountries[columns_to_select]
top_ruralcountries
```

	ISO3	Countries and areas	Rural (Residence)
22	DOM	Dominican Republic	1.00
55	SRB	Serbia	0.91
62	MKD	North Macedonia	0.90
64	TON	Tonga	0.90
33	JPN	Japan	0.83
51	RUS	Russian Federation	0.79
46	MNE	Montenegro	0.74
25	GEO	Georgia	0.72
2	ARM	Armenia	0.71
14	CHL	Chile	0.70

```
#top ten for urban
top_urbancountries = data.nlargest(10, 'Urban (Residence)')
columns_to_select = ['ISO3', 'Countries and areas', 'Urban (Residence)']
top_urbancountries = top_urbancountries[columns_to_select]
top_urbancountries
```

	ISO3	Countries and areas	Urban (Residence)
55	SRB	Serbia	0.97
51	RUS	Russian Federation	0.94
64	TON	Tonga	0.94
25	GEO	Georgia	0.93
62	MKD	North Macedonia	0.93
15	CHN	China	0.91
8	BRA	Brazil	0.89
14	CHL	Chile	0.89
2	ARM	Armenia	0.88
46	MNE	Montenegro	0.86

Task 2.3

Using the describe function on columns poorest and richest (wealth quintile) a range of statistics are outputted . Below shows the statistical measures printed out next to each other for easier

comparison. This is printed out as a percentage. Looking at the mean values 19.043% for the poorest and 61.69% for the richest shows a big separation between the two. To confirm the mean we also used the median as another statistical measure which calculated that Poorest (Wealth quintile) has a median of 20.888% and Richest (Wealth quintile) at 64.344%. With the similarity between mean and median it can be stated that the data does not have many outliers and the analysis of the difference between poorest and richest is reliable. Within the describe function we also have the std or standard deviation being at 26.63% for poorest and 33.17% For the Poorest (Wealth quintile) column as well as the Richest (Wealth quintile) with a standard deviation of 26.63% and 33.17% respectively, it indicates that the values within the columns have a significant amount of variability around the mean. This means that the values are more spread out from the average value.

```
wealth_stats = data[['Poorest (Wealth quintile)', 'Richest (Wealth quintile)']] * 100
wealth_stats = wealth_stats.describe().round(2)
wealth_stats
```

	Poorest (Wealth quintile)	Richest (Wealth quintile)
count	74.00	74.00
mean	19.03	61.69
std	26.63	33.17
min	0.00	0.00
25%	0.00	34.50
50%	6.00	67.00
75%	22.75	93.00
max	100.00	100.00

```
median = wealth_stats.median().round(3)
median
```

```
Poorest (Wealth quintile)    20.888
Richest (Wealth quintile)    64.344
dtype: float64
```

References

Pérez,F. (2011). Title of Jupyter Notebook. [Notebook]. <http://localhost:8888/tree>

Hou.M.(2023).week 3 practical data science lab.[activity.]

Gallagher.L(2023).week 5 advanced programming for data science lab slides.[slides]